

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE CIENCIAS
Compiladores

Práctica 4: Flex
Fast lexical analyzer generator

PROFESOR

Víctor Germán Mijangos de la Cruz

AYUDANTES

Alejandro Espino Gutiérrez
Itzel Paola Flores Carrillo
Adrián Martínez Manzo

ALUMNOS

Bravo Orozco Diego - 320222904
López Rodríguez Leslie Renée - 321171915
Raya García Marco Antonio - 422073121
Reyes Colín Luis Manuel - 321090474
Sautto Ramírez Seldon - 321084163

GRUPO 7008

30 de Septiembre de 2026

Índice

1. Estrategia de diseño léxico y condiciones de inicio	2
1.1. Lenguaje modelado	2
1.2. Organización de los archivos	2
1.3. Orden de las reglas	3
1.4. Condición de inicio para comentarios multilínea	3
1.5. Cadenas, caracteres y escapes	4
2. Instrucciones de compilación y ejecución de pruebas	4
2.1. Evidencia	4
3. Bitácora de casos límite y depuración léxica	6
3.1. Longest match en operadores compuestos	6
3.2. Varias cadenas en una línea se fusionaban	7
3.3. Salto de línea y cadenas sin cerrar	7
3.4. Ciclo infinito al no salir del estado exclusivo	7
3.5. Otros casos frontera	8
4. Limitaciones conocidas y deuda técnica	8
5. Validación en clase	9
6. Evaluación de la práctica	9

1. Estrategia de diseño léxico y condiciones de inicio

1.1. Lenguaje modelado

El lenguaje Minion conserva la estructura léxica de C (llaves, ;, operadores convencionales) y sustituye las palabras clave por vocabulario de los Minions. Los tipos, operadores y delimitadores mantienen su forma tradicional.

Lexema	Token	Equivalente
storia / filma	KW_STORY / KW_DEF	programa / definición de función
banana / gelato	KW_VAR / KW_CONST	variable / constante
int, float, string, char	KW_INT, ..., KW_CHAR	tipos de datos
para_tu / bi_do	KW_IF / KW_ELSE	if / else
stupa / hana_dul_sae	KW_WHILE / KW_FOR	while / for
luk_at_tu	KW_FOREACH	foreach
papoi	KW_RETURN	return
poopaye / pwde_na	KW_BREAK / KW_CONTINUE	break / continue
bello	KW_PRINT	función nativa print

Tabla 1: Palabras reservadas del lenguaje Minion.

Una declaración tiene la forma `banana int bananas = 3;`; la palabra de declaración y el tipo son tokens independientes y la validación de su combinación se deja al parser. Las palabras con guion bajo (`para_tu`, `hana_dul_sae`, ...) se reconocen como un solo lexema porque `_` pertenece al alfabeto de identificadores.

1.2. Organización de los archivos

- `scanner.h`: enum `ScannerToken` con `TOK_EOF = 0` (lo que `yylex` devuelve al terminar) y `TOK_ERROR = 256`; el resto toma valores desde 257, dejando libre el rango 1–255 que Bison usa para tokens de un carácter.
- `scanner.l`: sólo definiciones y reglas; cada acción se limita a `return TOK_...`
- `scanner.c`: el `main` llama a `yylex` hasta recibir `TOK_EOF` e imprime `[TOKEN:LEXEMA]` con `scanner_token_name` y `yytext`.

Código 1: Opciones, condición de inicio y definiciones regulares.

```
%option noyywrap yylineno
%x COMMENT

ID          [a-zA-Z_][a-zA-Z0-9_]*
INT         [0-9]+
FLOAT       ({INT}\. [0-9]*([eE] [+]?{INT})?|{INT}[eE] [+]?{INT})
WS          [ \t\r\n]+
```

`nookywrap` indica que hay una sola entrada y `yylineno` mantiene el número de línea. `FLOAT` acepta punto decimal con exponente opcional (2.5, 3., 1.5e-3) o exponente sin punto (1e5). `WS` incluye `\r` para aceptar archivos con finales de línea CRLF.

1.3. Orden de las reglas

Flex elige la regla con el lexema *más largo* y, si hay empate, la que aparece *primero*. El archivo se ordenó así:

1. **Comentarios primero.** `"/*` (2 caracteres) le gana a `"/` por longitud, y su posición documenta que tienen prioridad sobre cualquier otro uso de `/`.
2. **Palabras reservadas antes que {ID}.** Para `banana` ambas reglas empatan en longitud y gana la reservada (`KW_VAR`); para `bananas`, `{ID}` reconoce un carácter más y gana (`IDENTIFIER`). Si `{ID}` estuviera antes, ninguna palabra reservada se reconocería.
3. **Operadores compuestos antes que simples** (`>>`, `>=` antes que `>`). El longest match ya los resuelve, pero el orden refleja la prioridad real.
4. **{FLOAT} antes que {INT}**, para preferir la interpretación más específica.
5. **Cadena cerrada antes que cadena sin cerrar**; la válida siempre reconoce un carácter más.
6. **. al final** como comodín de errores: sólo aplica cuando nada más avanza y siempre consume un carácter, evitando el ECHO por defecto de Flex.

Código 2: Literales, identificadores y comodín de error.

```
{FLOAT}      { return TOK_FLOAT_LITERAL; }
{INT}        { return TOK_INT_LITERAL; }
\'([^\n]|\\.)\' { return TOK_CHAR_LITERAL; }
\"([^\n]|\\.)*\" { return TOK_STRING_LITERAL; }
\"([^\n]|\\.)*  { return TOK_ERROR; /* cadena sin cerrar */ }
{ID}         { return TOK_IDENTIFIER; }
<<EOF>>      { return TOK_EOF; }
.            { return TOK_ERROR; }
```

1.4. Condición de inicio para comentarios multilínea

Se usa la condición exclusiva `%x COMMENT:` con `%s` (inclusiva) las reglas de `INITIAL` seguirían activas y una palabra como `banana` dentro del comentario se reconocería como token.

Código 3: Reglas del estado COMMENT.

```
"/*"         { BEGIN(COMMENT); }
<COMMENT>"/" { BEGIN(INITIAL); }
<COMMENT>\n  { /* ylineno se actualiza automáticamente */ }
<COMMENT>.   { /* descartar contenido de comentario */ }
<COMMENT><<EOF>> { BEGIN(INITIAL); return TOK_ERROR; }
```

`<COMMENT>\n` y `<COMMENT>.` juntas cubren todos los caracteres, de modo que dentro del comentario siempre se consume algo. Con `**/`, el primer `*` lo consume `.` y después `"/` cierra. El fin de archivo dentro del comentario se reporta como error y regresa a `INITIAL`. Los comentarios de línea no necesitan estado: `"/" [^\n]*` los consume en una sola coincidencia.

1.5. Cadenas, caracteres y escapes

Las cadenas usan una sola expresión regular en lugar de un estado, porque el lexema se conserva tal como aparece en el fuente, sin decodificar. El cuerpo admite cualquier carácter excepto comilla, \ o salto de línea, o bien una secuencia \.. Así \" se consume como escape y no cierra la cadena. El literal de carácter usa el mismo cuerpo sin repetición ('M', '\n', '\n').

Los errores léxicos se reportan como [ERROR:lexema] en tres casos: un carácter fuera del alfabeto, una cadena sin cerrar al fin de línea y un comentario sin cerrar al fin de archivo. En todos el análisis continúa.

2. Instrucciones de compilación y ejecución de pruebas

Desde el directorio `src/` (Flex 2.6.4 y gcc):

```
flex scanner.l # genera lex.yy.c
gcc lex.yy.c scanner.c -o scanner

./scanner < ../tests/prueba_basica.minion
./scanner < ../tests/prueba_tokens.minion
./scanner < ../tests/prueba_errores.minion
./scanner < ../tests/prueba_casos_limite.minion
./scanner < ../tests/prueba_comentario_abierto.minion
```

Archivo	Qué valida	Tokens	Errores
prueba_basica	Programa completo: declaraciones, todas las estructuras de control, bello.	112	0
prueba_tokens	Literales y todos los operadores aritméticos, compuestos, relacionales, lógicos y de bits.	188	0
prueba_errores	@, #, ? y cadena sin cerrar.	27	4
prueba_casos_limite	Comentarios, notación científica, escapes, ++, &&&, +=, dos cadenas en una línea.	102	0
prueba_comentario_abierto	Comentario de bloque sin cerrar.	9	1

Tabla 2: Suite de pruebas. Entre todas se emiten los 56 tipos de token del lenguaje más ERROR.

2.1. Evidencia

Primeras líneas de `prueba_basica.minion` (banana es KW_VAR y bananas es IDENTIFIER):

```
storia minionAdventure {    -> [KW_STORY:storia] [IDENTIFIER:minionAdventure] [LBRACE:{]
  banana int bananas = 3;    -> [KW_VAR:banana] [KW_INT:int] [IDENTIFIER:bananas]
                              [ASSIGN:=] [INT_LITERAL:3] [SEMICOLON:;]
  gelato int maxBananas = 10; -> [KW_CONST:gelato] [KW_INT:int] [IDENTIFIER:maxBananas]
                              [ASSIGN:=] [INT_LITERAL:10] [SEMICOLON:;]
```

CAPTURA PENDIENTE

Salida de `./scanner < ../tests/prueba_basica.minion` donde se vean `KW_DEF`,
`KW_IF`, `KW_ELSE`, `KW_WHILE`, `KW_FOREACH`, `KW_BREAK`,
`KW_CONTINUE` y `KW_RETURN`.
(figuras/prueba_basica.png)

Figura 1: Ejecución de `prueba_basica.minion`.

CAPTURA PENDIENTE

Salida de `./scanner < ../tests/prueba_tokens.minion` con `FLOAT_LITERAL`,
`KW_FOR`, asignaciones compuestas y `SHL/SHR/BIT_AND/BIT_OR`.
(figuras/prueba_tokens.png)

Figura 2: Ejecución de `prueba_tokens.minion`: literales y operadores.

Errores léxicos en `prueba_errores.minion`:

```
x = x @ 2;          -> [IDENTIFIER:x] [ERROR:@] [INT_LITERAL:2]
x = x # 3;          -> [IDENTIFIER:x] [ERROR:#] [INT_LITERAL:3]
x = x ? 4;          -> [IDENTIFIER:x] [ERROR:?] [INT_LITERAL:4]
bello("cadena sin cerrar); -> [KW_PRINT:bello] [LPAREN:()] [ERROR:"cadena sin cerrar);]
```

CAPTURA PENDIENTE

Salida completa de `./scanner < ../tests/prueba_errores.minion` con los cuatro
ERROR.
(figuras/prueba_errores.png)

Figura 3: Ejecución de `prueba_errores.minion`.

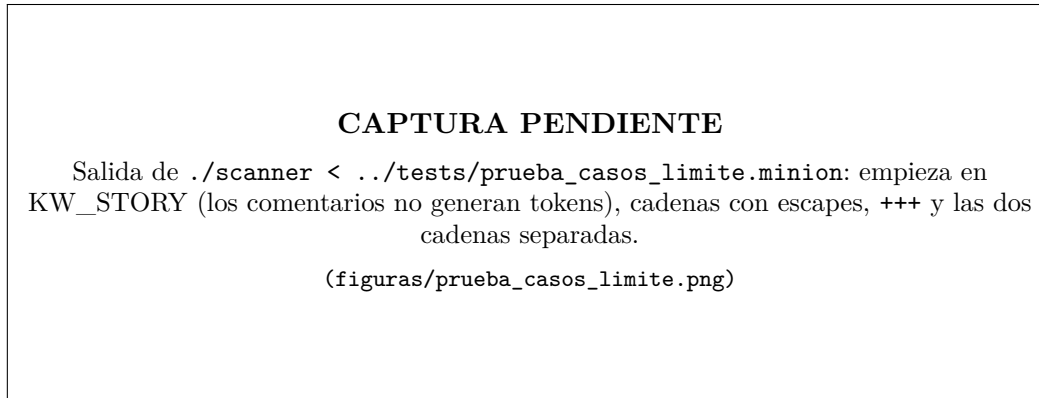


Figura 4: Ejecución de `prueba_casos_limite.minion`.

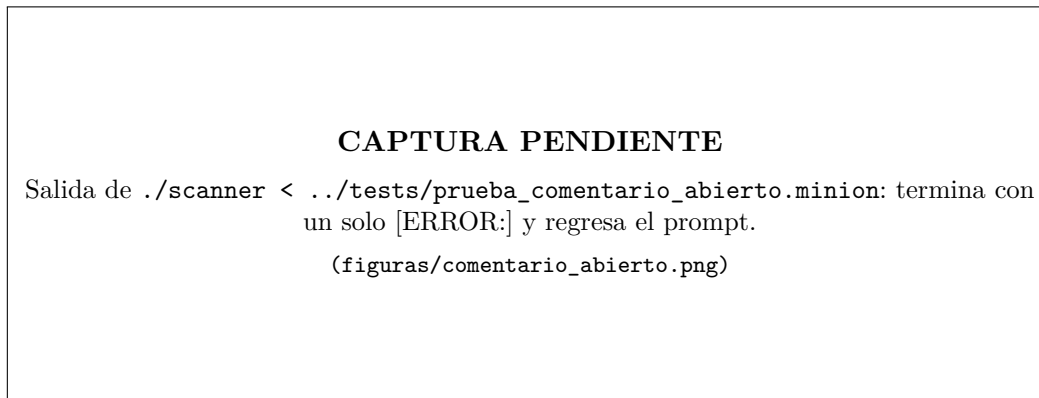


Figura 5: Comentario sin cerrar: un error y el scanner termina.

3. Bitácora de casos límite y depuración léxica

3.1. Longest match en operadores compuestos

Con operadores que son prefijos de otros (`>`, `>=`, `>>`; `+`, `++`, `+=`; `&`, `&&`), Flex elige la versión compuesta sin *lookahead* manual:

Entrada	Tokens	Explicación
<code>a+++b</code>	ID INC PLUS ID	Primero <code>++</code> (el más largo); el tercer <code>+</code> queda solo.
<code>&&& / </code>	AND BIT_AND / OR BIT_OR	Mismo principio.
<code>a>>2<<1</code>	ID SHR INT SHL INT	<code>>></code> y <code><<</code> ganan a <code>></code> y <code><</code> .
<code>a<=<b</code>	ID SHL ASSIGN ID	No existe <code><=<</code> ; desde <code><</code> lo más largo es <code><<</code> .

Tabla 3: Secuencias de operadores ambiguas.

También se verificó que `bananas`, `para_tuyo` e `integer` son IDENTIFIER (no se reconoce la palabra reservada por prefijo) y que `Storia` también lo es, pues las palabras reservadas distinguen mayúsculas.

3.2. Varias cadenas en una línea se fusionaban

Problema. La regla original no excluía la comilla del cuerpo de la cadena:

```
\"([^\\"\\n]|\\.)*\"      { return TOK_STRING_LITERAL; }
```

Por longest match, la cadena se extendía hasta la última comilla de la línea:

```
bello("primera", "segunda"); -> [STRING_LITERAL:"primera", "segunda"]
```

Las pruebas originales no lo detectaban porque ninguna línea tenía dos cadenas. Los `char` tenían el mismo defecto.

Solución. Se agregó la comilla a la clase negada: `[^"\\n]` para cadenas y `[^'\\n]` para caracteres. La comilla escapada se sigue aceptando mediante `\\.`

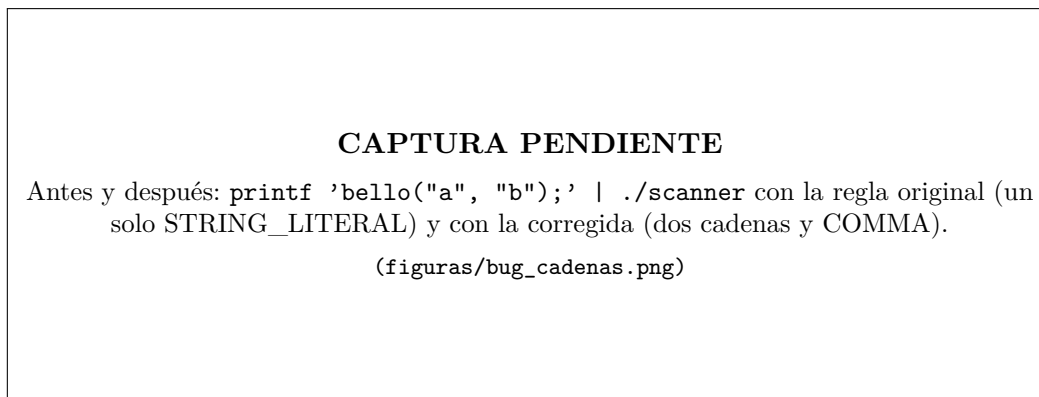


Figura 6: Fusión de cadenas antes y después de la corrección.

3.3. Salto de línea y cadenas sin cerrar

Las cadenas no pueden abarcar varias líneas, por eso `\n` está excluido del cuerpo. Sin una regla específica, una cadena sin cerrar dejaba la comilla en el comodín `.` y el resto se tokenizaba como código (`[ERROR:] [IDENTIFIER:cadena] [IDENTIFIER:sin] ...`), generando tokens falsos. Se agregó la regla `\"([^\\"\\n]|\\.)*`, que se detiene antes del salto de línea y reporta el resto de la línea como un solo error. Cuando la cadena sí cierra, la regla válida reconoce un carácter más y gana.

3.4. Ciclo infinito al no salir del estado exclusivo

Problema. La regla original era `<COMMENT><EOF>> { return TOK_ERROR; }`. Al devolver el error, el scanner seguía en `COMMENT`; la siguiente llamada a `yylex` volvía a encontrar el fin de archivo en ese estado y devolvía otro error, sin llegar nunca a `TOK_EOF`. Con la entrada `x /* sin cerrar`, en dos segundos se imprimieron más de 3.8 millones de líneas `[ERROR:]`.

Solución. Agregar `BEGIN(INITIAL);` antes del `return`. En la siguiente llamada se aplica la regla `<EOF>>` general y el ciclo termina tras un único error (Figura 5).

CAPTURA PENDIENTE

Con la regla original: `printf 'x /* sin cerrar' | timeout 2 ./scanner | head`
mostrando `[ERROR:]` repetido. Con la corrección: un solo `[ERROR:]`.
(`figuras/bug_loop.png`)

Figura 7: Ciclo infinito en el estado `COMMENT`, antes y después de la corrección.

Por la misma razón, el estado `COMMENT` necesita `<COMMENT>\n` además de `<COMMENT>..` Sin esa regla, los saltos de línea no coincidirían con nada y Flex los copiaría a la salida (`ECHO`). Además, no se usa ningún patrón que acepte la cadena vacía, porque eso provocaría un ciclo sin consumir caracteres.

3.5. Otros casos frontera

- **Comentarios dentro de cadenas.** Una cadena que contiene `/*` o `//` sigue siendo un solo `STRING_LITERAL`: la regla de cadena reconoce todo desde la comilla y Flex no reanaliza el interior de un lexema.
- **Números.** `1.2.3` produce `FLOAT(1.2) ERROR(.) INT(3)`; `1e` produce `INT(1) ID(e)`, porque Flex retrocede a la última regla aceptada; `.5` produce `ERROR(.) INT(5)`.
- **CRLF.** Sin `\r` en `WS`, cada línea de un archivo editado en Windows terminaría en un `[ERROR:]` aparentemente vacío.

4. Limitaciones conocidas y deuda técnica

- **UTF-8.** Flex trabaja byte por byte: ñ produce dos `[ERROR:...]` con bytes sueltos. No hay identificadores con acentos; sí se aceptan dentro de cadenas y comentarios.
- **Literales numéricos.** Hay notación científica, pero no hexadecimales (`0x1F` da `INT(0) ID(x1F)`), octales, `.5` ni sufijos. `123abc` se acepta como dos tokens en lugar de reportarse como error.
- **Operadores de bits.** Faltan `^` y `~` (hoy son `ERROR`) y las asignaciones `<<=`, `&=`, etc.
- **Comentarios anidados.** `/* a /* b */ c */` cierra en el primer `*/`; requeriría un contador de profundidad.
- **Escapes.** No se decodifican ni se validan (`\q` se acepta). Tampoco hay cadenas multilínea.
- **Ubicación de errores.** Aunque `yylineno` está activo, no se imprime la línea; el comentario sin cerrar se reporta con lexema vacío.

- **Indentación y preprocesador.** No se emiten INDENT/DEDEDENT (el lenguaje usa llaves) ni se procesan directivas `#include`; esto requeriría pila de niveles y búferes múltiples en `scanner.c`.

5. Validación en clase



FOTOGRAFÍA DEL EJERCICIO VALIDADO EN CLASE
(figuras/validacion.jpg)

Validación: Luis Manuel Reyes Colín

6. Evaluación de la práctica

Escala: 1. Totalmente en desacuerdo, 2. En desacuerdo, 3. Neutral, 4. De acuerdo, 5. Totalmente de acuerdo.

Luis

Criterio / Afirmación	1	2	3	4	5
Los conceptos y habilidades desarrollados en esta práctica son relevantes para mi formación profesional.		*			
El trabajo realizado me permitió comprender de forma práctica la teoría revisada en clase.			*		
Las actividades y problemáticas planteadas durante la sesión resultaron atractivas y motivadoras.			*		
El enfoque práctico y los retos propuestos captaron mi interés a lo largo del desarrollo.			*		
El nivel de complejidad técnica fue adecuado respecto a los conocimientos y recursos previos disponibles.			*		
La claridad de la guía permitió realizar la práctica de manera fluida y sin bloqueos innecesarios.			*		